

Програмування мовою C++. Функції в C++.

1. ФУНКЦІЇ В C++

1.1. Поняття функції

Функція — це блок коду, який виконує певну задачу.

Виклик функції **перериває** виконання поточної, виконує іншу, а потім повертається назад.

Формат:

```
тип_повернення ім'я(параметри) {  
    // тіло функції  
}
```

1.2. Значення, що повертаються

- Функція може **повертати одне значення** через return.
- Тип повернення вказується перед ім'ям.
- void — функція нічого не повертає.

```
int f() { return 7; }  
void g() { /*...*/ }
```

1.3. Прототип функції

Компілер читає зверху вниз.

Якщо виклик функції стоїть вище її визначення — потрібен **прототип**.

```
int add(int a, int b); // прототип
```

```
int main() { add(3,4); }
```

```
int add(int a, int b) { return a+b; }
```

1.4. Параметри та аргументи

- **Аргументи** — значення, що передаються при виклику.
- **Параметри** — змінні всередині функції.

```
void f(int x, int y) { } // x, y – параметри  
f(3, 4);                // 3, 4 – аргументи
```

2. СПОСОБИ ПЕРЕДАЧІ ПАРАМЕТРІВ

В C++ є 3 способи:

1. Передача по значенню
2. Передача по посиланню
3. Передача за адресою (через вказівник)

2.1. Передача по значенню

Передається **копія** аргументу.

```
void f(int x) { x = 10; }  
int a = 5; f(a); // a не зміниться
```

2.2. Передача по посиланню

Параметр — **псевдонім** аргументу.

```
void f(int& x) { x = 10; }  
int a = 5; f(a); // a = 10
```

Константне посилання

Якщо функція **не повинна змінювати** аргумент:

```
void f(const Big& obj);
```

2.3. Передача за адресою (вказівник)

```
void f(int* p) { *p = 10; }  
int a = 5; f(&a); // a = 10
```

3. ПОСИЛАННЯ (REFERENCES)

Основні властивості:

- це **псевдонім** іншої змінної
- потребує **обов'язкової ініціалізації**
- не може бути перенаправлене
- не може бути nullptr
- доступ до значення — як до звичайної змінної

```
int x = 5;  
int& r = x;  
r = 7; // x = 7
```

Типи посилань

1. **lvalue reference**: int& a
2. **const reference**: const int& b
3. **rvalue reference (C++11)**: int&& c (пізніше)

Посилання vs. rvalue

Неконстантне посилання НЕ може бути прив'язане до rvalue:

```
int& r = 5;    // помилка
```

```
const int& r = 5; // дозволено
```

4. ВКАЗІВНИКИ (POINTERS)

Вказівники **містять адресу** об'єкта.

```
int x = 5;  
int* p = &x;  
*p = 7; // x = 7
```

Властивості:

- можуть бути nullptr
- можуть передавати на інший об'єкт
- вимагають * (розіменування) та & (адреса)

5. РІЗНИЦЯ МІЖ ПОСИЛАННЯМИ ТА ВКАЗІВНИКАМИ

Характеристика	Посилання	Вказівник
Можливість бути порожнім	✗	✓
Потрібна ініціалізація	✓ (обов'язково)	✗
Можна змінити "на що вказує"	✗	✓
Розіменування	не потрібно	потрібно *
Безпечність	більш безпечні	менш безпечні
Застосування	параметри, повернення	динамічна пам'ять, nullable

6. Повернення значень

6.1. Повернення по значенню

Створюється **копія**:

```
int f() { int x = 5; return x; }
```

Коли використовувати:

- повернення локальних змінних
- повернення невеликих об'єктів

6.2. Повернення по посиланню

Повернення існуючого об'єкта, який не зникне з пам'яті.

```
int& foo() {  
    static int x = 5;  
    return x;  
}
```

Не можна повертати:

- локальні змінні, бо вони знищуються

6.3. Повернення за адресою

Корисно для динамічної пам'яті.

```
int* f() { return new int(5); }
```

7. ТАБЛИЦЯ РЕКОМЕНДАЦІЙ (КОРОТКО)

(Створено на основі файлу з таблицею)

Передача параметрів

Спосіб	Коли використовувати	Коли НЕ використовувати
по значенню	фундаментальні типи	великі об'єкти
по посиланню	коли треба змінити аргумент або уникнути копії	передача масивів? → краще pointer
за адресою	масиви, nullable	фундаментальні типи, класи

Повернення значень

Спосіб	Коли використовувати	Не використовувати
по значенню	локальні змінні	великі структури
по посиланню	повернення параметру або глобальної змінної	локальні змінні
за адресою	динамічна пам'ять	локальні змінні